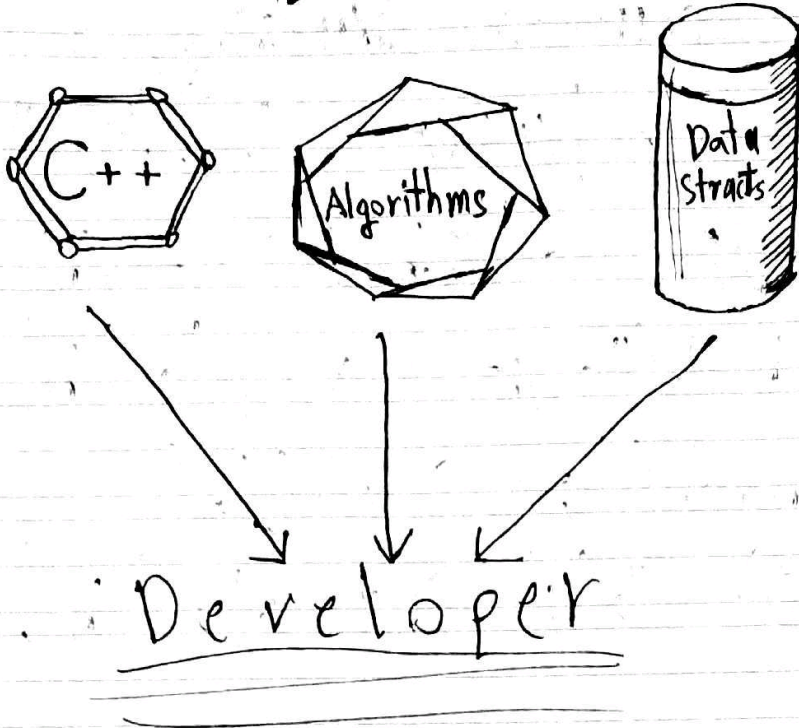


C++ DSA 1

coder → to



You need to:

1. have a good knowledge in c++.
2. know how to use c++.

8/9/2025.

George Tomeh

(2)

Date: / /



Subject:

array: { 0 1 2 3 4 5 6 7 } static size
accessible by index

Vector

Vectors: `std::vector<std::string> x = {"a", "b"};`

`<x[0]; ==> a` `<x[1]; ==> b` `<x.at(i) ==>` ^{stop error} _{don't show error}

`<x.front() ==> a` `<x.back() ==> b`

`<x.push-back("c"); ==> x = {"a", "b", "c"};`

`<x.pop-back(); ==> x = {"a"};`

`x.at(0) = "d"; ==> x = {"d", "b"};`

closed open
push-front() pop-front() `a` `b` `c` `d` `e` `f` `g` `h` `i` `j` `k` `l` `push-back();` `pop-back();`
front access `[0]; = .at(0);` back access
get set get set

`.size();`
`.empty();`

`insert(index, n, value)`
optional

Sabbagh

3

Date: / /



Subject:

List

You can use:

```
for (std::string f: listOne)
```

```
{
```

```
    std::cout << f << '\n';
```

```
}
```

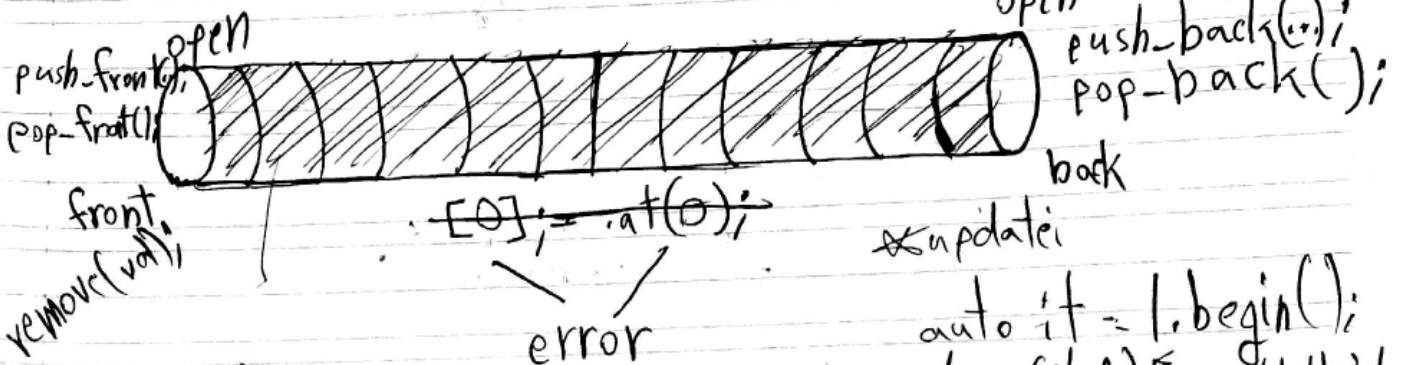
```
std::list<std::string> x = {"a", "b", "c", "d"};
```

like vector but

1- can make:
- push-front(...);
- push-back(...);

- pop-front();
- pop-back();

2- doesn't support random access; you can't use [] or .at();



.size();
.empty();

~~you can change the~~ insert(it, val);

you can move the iterator using the advance() function

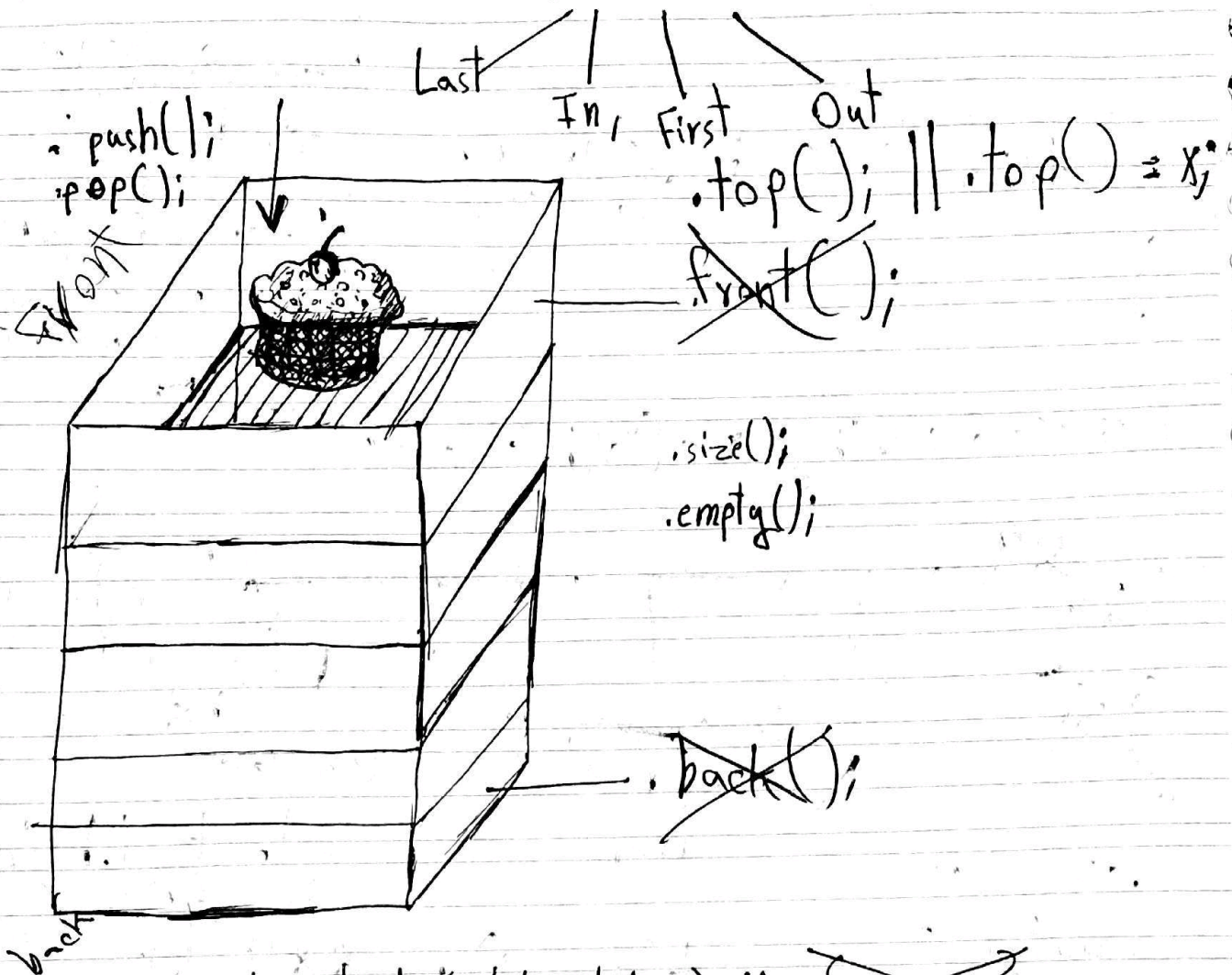
~~update~~
auto it = l.begin();
advance(it, 2) // move it to the 3rd element
~~it = 10;~~
// changing the value by dereferencing the it

```
erase(it);  
auto it = l.begin();  
advance(it, 2);  
erase(it);  
// remove the third element
```

Sabbagh

Stack

A stack stores multiple elements in a specific order called LIFO:



`std::stack<std::string> s;`

Queue

A Queue stores multiple elements in a specific order called

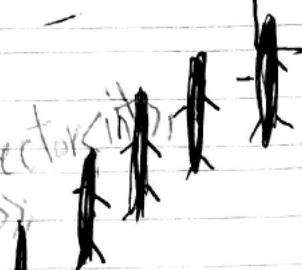


~~[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]~~

.size();
.empty();



front();
pop();

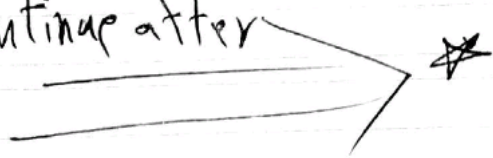


back();
push();

using namespace std;

~~queue<string> x; = {"a", "b", "c"}~~

Continue after



6

Date: / /



Subject:

React APIs

~~without save~~

~~with save~~

1. import {useEffect, useState} from 'react';

2. import type {user} from 'api/user';

3. const user = () => {

4. const [user, setUser] =
useState(null)

5.

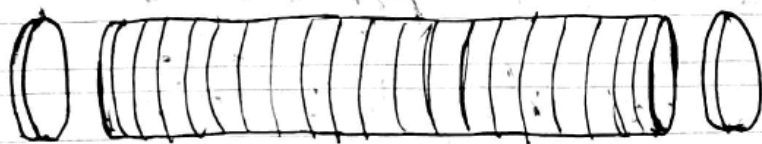
6. useEffect

Deque

A deque (stands for double-ended queue) however, is more flexible, as elements can be added and removed from both ends (at the front and at the back), and you can also access elements by index numbers.

back push-back(...);
pop-back();

push-front(...);
pop-front();
front



magnet Queue is considered a Deque

.empty();

.size();

[0] || .at(0)

for (string i : dequeOne) { ... }

.at(...); not [] because .at(...); throws an error

std::deque<string> x = { ... }

Set

A set stores unique elements where they:

- Are stored automatically in ascending order
- Are unique, meaning equal or duplicate values are ignored
- Can be added or removed, but the value of an existing element cannot be changed.
- Cannot be accessed by index numbers, because the order is based on sorting and not indexing.

```
std::set<std::string> x = {"a", "b", "c"};
```

```
for (std::string & x) {<< i<<}
```

Sorted

for descending order

you can loop through using iterator

use:

```
std::set<std int, std::greater<int>> n = {0, 1, 2};
```

```
.empty();
```

```
.size();
```

```
.insert(...);
```

add elements

specific

```
.erase(...);
```

remove specific element

```
.clear();
```

remove all elements

⑨

Map

A map stores elements in "key/value" pairs.

Elements in a map are:

- Accessible by keys (not index), and each key is unique.
- Automatically stored in ascending order by their keys.

using namespace std;

map<string, int> x = {{"a", 13}, {"b", 23}};

You cannot use ~~[]~~ or ~~at()~~ like arrays and

vectors. Instead you can access a map element by referring to its key inside square brackets `[ ]` or `at()`.

You can also change the value associated with a key.

You can add element using square brackets `[ ]`:

thisMap["new key"] = "the value";

and you can also use the `.insert(...)`;

~~* insert({"d", 4})~~

you can check if a specific element exists using `count("key")`;

~~reverse()~~
~~clear()~~

Date

Note: if you want reverse order you can use
greater<type> inside the angle brackets, like:
map<string, int, greater<string>> people = {{}};

②

looping through a map

for-each loop

- You should use the auto keyword inside for loop. This allows the compiler to determine automatically the correct datatype for each key-value pair.
- Since map elements consists of both keys and values, you have to include .first to access the keys and .last to access the values in the loop.
- Elements in the map are sorted automatically in ascending order by their keys.

eg: map<string, int> people = {{ "John", 32 }, { "Adele", 45 }, { "Bo", 29 } };

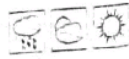
```
for(auto person : people) {
```

```
    cout << person.first << " is " << person.second << "\n";
```

```
}
```

It is possible to loop through using iterator.

Sabbagh



Subject:

Iterators

Iterators are used to access and iterate through elements of data structures (vectors-sets-etc.) by "pointing" them

It is called "iterator" because "iterating" is the technical term for looping.

let's iterate through a vector:

```
vector<string> cars = {"Volvo", "BMW", "Ford",  
                        "Mazda"};
```

```
vector<string>::iterator it;
```

```
for(it = cars.begin(); cars it != cars.end(); ++it)
```

```
{
```

```
    *it  
    cout << it << '\n';
```

```
}
```



Subject: _____

Explain:

1. First, we create a vector of strings to store the names of different car manufactures.

Lost
Data

3

4

5

`begin()` and `end()` belongs to vector not to iterator. `begin()` ^{points to the} $\Rightarrow$ first element

`begin() + 1` ^{points to the} $\Rightarrow$ second element

`end()` points to one position after the last element
`end() - 1` $\Rightarrow$ last element

Sabbagh



Subject:

You can modify the same data instead of modifying a copy of it.

eg: `*it = "...";`

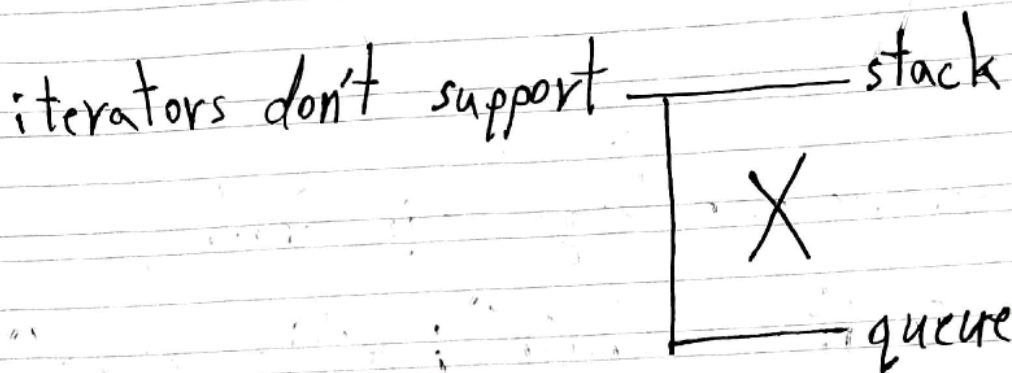
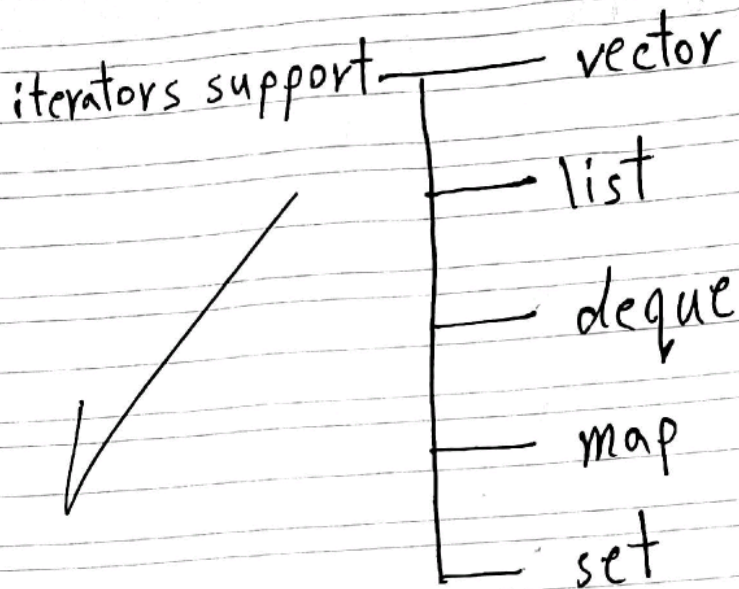
Lost Data

You can also use `auto` keyword in for loops

```
for(auto it = cars.begin(); it != cars.end(); ++it)
{ cout << *it << '\n'; }
```



*Note: in pointers we use $(\rightarrow)$ instead of $(.)$.
we use it with iterators too (map iterators).



use `rbegin()` and `rend()` to iterate in reverse.

algorithms with iterators:

Another important feature of iterators is that they are used with different algorithm functions, such as `sort()` and `find()` (found in the `<algorithm>` library) to sort and search for elements in a data structure.

Example:

1. `#include <iostream>`

2. `#include <vector>`

3. `#include <algorithm>`

4. `using namespace std;`

5. `int main() {`

6. `vector<string> alpha = {"b", "a", "d", "c"};`

7. `sort(alpha.begin(), alpha.end());`

8. `for (string letter : alpha) { cout << letter << '\n'; }`

9. `return 0; }`



Subject: _____

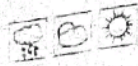
sort in reverse

Example 1

```
1. #include <bits/stdc++.h>
2. using namespace std;
3. int main() {
4.     vector<int> nums = {3, 6, 8, 1, 2, 5, 4, 7, 0};
5.     sort(nums.rbegin(), nums.rend());
6.     for (int num : nums) {
7.         cout << num << " ";
8.     }
9.     return 0;
10. }
```

search using `find(x.begin(), x.end(), "d");`

like `auto iterator = find(~~, ~~, ~~);`



to search for the first element that is greater than a specific value; you can use the `upper_bound()`; ~~find~~ function.

like:

* but first you have to ~~find~~ sort the DS *

```
vector<int> nums = {1, 7, 3, 5, 9, 2};
sort(nums.begin(), nums.end());
```

```
auto it = find upper_bound(nums.begin(), nums.end(), 5);
```

You can use `min_element()` function, to find the smallest element or `max_element()`, to find the largest element.

like:

```
vector<int> nums = {1, 7, 3, 5, 9, 2};
```

```
auto it = min_element(nums.begin(), nums.end());
```



Date: / /



Subject:

to copy elements from one vector to another,
you can use the `copy()` function:

like

```
vector<int> nums = { 1, 4, 3, 5, 9, 2 };
```

```
vector<int> copiedNums(6);
```

```
copy(nums.begin(), nums.end(), copiedNums.begin());
```

to fill all elements in a vector with a value,
you can use the `fill()` function:

like:

```
vector<int> nums(6);
```

```
fill(nums.begin(), nums.end(), 3);
```

Date

(19)

Subject

* Luhn algorithm

Luhn algorithm: to validate a credit card number:

- ① double every second digit from right to left
if doubled number is two digits, split them.
- ② add all single digits from ~~right to left~~ step ①.
- ③ add all odd numbered digits from right to left.
- ④ Sum results from steps ② & ③
- ⑤ if step ④ is divisible by 10, the number is valid.

Date

20

Geo

Subject